

Теория

Основы языка Go

1. Какие основные принципы и философия языка Go?

Go создан с фокусом на простоту, читаемость и продуктивность разработки. Ключевые принципы: - **Простота**: минималистичный синтаксис, отсутствие избыточных возможностей - **Явность**: код должен быть понятен без глубокого знания языка - **Компиляция**: статическая типизация и быстрая компиляция - **Конкурентность**: встроенная поддержка горутин и каналов - **Производительность**: близкая к C производительность при простоте использования - **Стандартизация**: единый стиль кода через `gofmt` и `golint` - **Одна задача - один способ**: избегание множественных способов решения одной задачи

2. В чем разница между `var`, `:=` и `new()` ?

- `var`: объявляет переменную с zero value типа. Можно использовать вне функций, позволяет указать тип явно: `var x int` или `var x = 10`
- `:=`: короткое объявление переменной с автоматическим выводом типа. Работает только внутри функций. Одновременно объявляет и инициализирует: `x := 10`
- `new()`: выделяет память в куче, инициализирует zero value и возвращает указатель на тип: `x := new(int)` эквивалентно `var x *int = new(int)`, где `*x == 0`

3. Что такое zero value в Go и для каких типов?

Zero value — значение по умолчанию для переменной, объявленной без инициализации: - Числовые типы: `0` (int, float, complex) - Булевы: `false` - Строки: `""` (пустая строка) - Указатели, слайсы, каналы, интерфейсы, функции: `nil` - Структуры: все поля имеют zero value - Массивы: все элементы имеют zero value

4. Как работает `defer` и в каком порядке выполняются `defer`-вызовы?

`defer` откладывает выполнение функции до момента возврата из текущей функции. Выполняется даже при панике или раннем `return`. Порядок выполнения: LIFO (Last In, First Out) — последний `defer` выполняется первым. Аргументы функции в `defer` вычисляются немедленно, но сама функция вызывается позже.

5. Что такое `init()` функция и когда она вызывается?

`init()` — специальная функция, автоматически вызываемая перед `main()`. В пакете может быть несколько `init()` функций — они выполняются в порядке объявления. Вызывается один раз при импорте пакета. Используется для инициализации пакета, регистрации компонентов, проверки зависимостей.

6. Как работает `recover()` и где его можно использовать?

`recover()` останавливает панику и возвращает значение, переданное в `panic()`. Работает только внутри `defer` функций. Если паники нет, возвращает `nil`. Используется для обработки критических ошибок, логирования паник, graceful degradation. Нельзя использовать для обычной обработки ошибок — только для восстановления после паники.

7. Что такое `iota` и как она работает?

`iota` — предопределенный идентификатор для создания последовательных констант. В каждом `const` блоке начинается с 0 и инкрементируется на 1 для каждой константы. Сбрасывается в каждом новом `const` блоке. Используется для enum-подобных констант, битовых масок, вычисляемых констант.

8. Как устроены слайсы (slices) в Go? Что такое capacity и length?

Слайс — это дескриптор массива, состоящий из трех полей: указатель на базовый массив, длина (length) и емкость (capacity). Length — количество элементов в слайсе, capacity — размер базового массива. Слайс не хранит данные, а ссылается на часть массива. При создании через `make([]T, len, cap)` или литерал `[]T{...}` создается базовый массив.

9. Как работает функция `append()` и когда происходит переаллокация?

`append()` добавляет элементы в конец слайса. Если capacity достаточна, элементы добавляются в существующий массив. При нехватке места происходит переаллокация: создается новый массив большего размера (обычно в 2 раза), данные копируются, возвращается новый слайс. Старый массив остается в памяти до сборки мусора. Переаллокация происходит при `len(slice) == cap(slice)`.

10. Как устроена `map` в Go? Какая структура данных лежит в основе?

Map реализована как хеш-таблица. Внутри содержит массив бакетов (buckets), каждый бакет — связанный список пар ключ-значение. При добавлении элемента вычисляется хеш ключа, определяется бакет, элемент добавляется в список бакета. При переполнении (load factor > 6.5) происходит рехеширование — создается новая таблица в 2 раза больше, все элементы перераспределяются.

11. Как решаются коллизии в `map`?

Коллизии решаются методом цепочек (chaining). Каждый бакет содержит связанный список пар ключ-значение. При коллизии (одинаковый хеш) новый элемент добавляется в список того же бакета. Поиск проходит по списку бакета, сравнивая ключи напрямую после проверки хеша.

12. Какая сложность операций `get`, `put`, `delete` в `map`?

В среднем $O(1)$, в худшем случае $O(n)$ при вырождении всех элементов в один бакет (крайне редко). На практике операции близки к $O(1)$ благодаря равномерному распределению хешей и автоматическому рехешированию при переполнении.

13. Что такое `sync.Map` и когда его использовать вместо обычной `map`?

`sync.Map` — потокобезопасная реализация `map` из пакета `sync`. Используется когда: много горутин читают, но редко пишут; ключи стабильны во времени; нужна потокобезопасность без внешних мьютексов. Не подходит для частых записей — обычная `map` с `sync.RWMutex` будет эффективнее. `sync.Map` оптимизирована для read-heavy сценариев.

14. Почему нельзя использовать `map` как ключ в другой `map`?

Ключи `map` должны быть сравнимыми типами (comparable). `map` не является comparable типом, так как сравнение `map` требует сравнения всех элементов, что не определено в Go. Только базовые типы, указатели, структуры и массивы без `map/slice/func` полей могут быть ключами.

15. В чем разница между массивом и слайсом?

Массив — фиксированный размер, тип включает размер: `[5]int`. При передаче копируется целиком. Слайс — динамический, тип без размера: `[]int`. При передаче копируется только дескриптор (указатель, len, cap). Массивы используются редко, слайсы — основной способ работы с последовательностями.

16. Что произойдет при копировании слайса? Копируются ли элементы?

При присваивании `s2 := s1` копируется только дескриптор слайса (указатель, len, cap). Элементы не копируются — оба слайса ссылаются на один базовый массив. Изменения через один слайс видны через другой. Для копирования элементов используется `copy(dst, src)` или `append([]T{}, src...)`.

17. Что из себя представляют строки в Go? Как они устроены?

Строка в Go — это неизменяемая (immutable) последовательность байтов. Внутри строка представлена структурой из двух полей: указатель на массив байтов и длина (length). Строки хранятся в UTF-8 кодировке. При присваивании `s2 := s1` копируется только дескриптор (указатель + длина), данные не копируются. Строки нельзя изменять напрямую — любая операция создает новую строку. Для модификаций эффективнее использовать `strings.Builder` или `[]byte`. При итерации через `range` происходит декодирование UTF-8 в руны, при индексации `s[i]` — доступ к байтам.

Горутины и конкурентность

17. Что такое горутина и чем она отличается от системного потока?

Горутина — легковесный поток выполнения, управляемый runtime Go. Отличается от системного потока: меньший размер стека (2KB vs 1-2MB), быстрое создание/переключение, управление планировщиком Go, а не ОС. Множество горутин выполняется на меньшем количестве системных потоков (M:N модель). Переключение между горутин дешевле, чем между потоками ОС.

18. Как работает планировщик горутин (Goroutine Scheduler) в Go?

Планировщик использует M:N модель: M горутин выполняются на N системных потоках ($M \gg N$). Компоненты: M (machine) — системные потоки, G (goroutine) — горутины, P (processor) — контексты процессоров. P содержит локальную очередь горутин. При блокировке горутины (syscall, channel) M передает P другому потоку. Work-stealing: простаивающий P может "украсть" горутину из другой очереди.

19. Что такое M:N модель и как она реализована в Go?

M:N модель — множество пользовательских потоков (горутин) выполняется на меньшем количестве системных потоков.

Преимущества: быстрое переключение контекста, эффективное использование CPU, масштабируемость. В Go: планировщик управляет распределением горутин по системным потокам, автоматически балансирует нагрузку, обрабатывает блокирующие операции без блокировки всего потока.

20. Что такое context и для чего он используется?

`context.Context` — интерфейс для передачи сигналов отмены, таймаутов и значений между горутин. Используется для: отмены долгих операций, установки дедлайнов, передачи request-scoped данных. Создается через `context.Background()`, `context.TODO()`, `context.WithCancel()`, `context.WithTimeout()`. Должен передаваться как первый параметр функции.

21. Как правильно отменять горутины через context?

Создать контекст с отменой: `ctx, cancel := context.WithCancel(parent)`. Передать `ctx` в горутину. В горутине проверять `ctx.Done()` в цикле или использовать `select` с `case <-ctx.Done()`. Вызвать `cancel()` для отмены всех дочерних операций. Контекст распространяет отмену вниз по дереву вызовов.

22. Что такое context.WithTimeout и context.WithCancel?

`context.WithTimeout(ctx, duration)` создает контекст с автоматической отменой через указанное время.

`context.WithCancel(ctx)` создает контекст с ручной отменой через `cancel()`. Оба возвращают дочерний контекст и функцию отмены. Отмена родительского контекста автоматически отменяет дочерние.

23. Что такое context.WithValue и когда его использовать?

`context.WithValue(ctx, key, value)` добавляет значение в контекст. Используется для передачи request-scoped данных (request ID, user info, trace ID). Ключи должны быть сравнимыми, рекомендуется использовать кастомные типы. Не использовать для передачи опциональных параметров — это антипаттерн. Только для данных, которые должны пройти через весь call chain.

24. Что такое sync.WaitGroup и как его использовать?

`sync.WaitGroup` — счетчик для ожидания завершения группы горутин. Методы: `Add(delta)` — увеличивает счетчик, `Done()` — уменьшает на 1 (вызывать в defer), `Wait()` — блокируется до обнуления счетчика. Используется для синхронизации: запустить N горутин, дождаться их завершения перед продолжением.

25. В чем разница между sync.Mutex и sync.RWMutex?

`sync.Mutex` — обычный мьютекс, один писатель или один читатель. `sync.RWMutex` — read-write мьютекс: множественные читатели (`RLock()`) или один писатель (`Lock()`). `RWMutex` эффективнее при read-heavy нагрузке, так как читатели не блокируют друг друга. Писатель блокирует всех читателей и других писателей.

26. Что такое sync.Once и когда его применять?

`sync.Once` гарантирует однократное выполнение функции через `Do(f)`. Используется для: ленивой инициализации сингلتонов, одноразовой настройки, инициализации глобальных ресурсов. Потокбезопасен, эффективнее чем мьютекс для одноразовых операций.

27. Что такое sync.Cond и для чего она нужна?

`sync.Cond` — условная переменная для синхронизации горутин по условию. Методы: `Wait()` — ждет сигнала, `Signal()` — будит одну горутину, `Broadcast()` — будит все. Используется когда горутины должны ждать изменения состояния, а не просто завершения других горутин. Требуется связанный мьютекс.

28. Что такое atomic операции и когда их использовать?

atomic операции — атомарные операции над переменными из пакета `sync/atomic`. Гарантируют thread-safe доступ без мьютексов для простых операций (инкремент, сравнение-и-замена, загрузка/сохранение). Используются для: счетчиков, флагов, lock-free структур данных. Быстрее мьютексов для простых операций, но ограничены базовыми типами.

29. Что такое data race и как его обнаружить?

Data race — ситуация, когда несколько горутин одновременно обращаются к одной переменной, и хотя бы одна записывает. Может привести к неопределенному поведению. Обнаруживается: `go run -race`, `go test -race`, `go build -race`. Race detector отслеживает доступы к памяти и сообщает о потенциальных гонках. Замедляет выполнение в 2-20 раз.

30. Как работает `-race` флаг в Go?

Race detector инструментирует код для отслеживания доступа к памяти. Во время выполнения отслеживает: какие горутин обращаются к памяти, в каком порядке, есть ли синхронизация. При обнаружении потенциальной гонки выводит отчет с `stack trace`. Используется только для тестирования и разработки, не для production из-за накладных расходов.

31. Что такое `goroutine leak` и как его предотвратить?

Goroutine leak — ситуация, когда горутина продолжает работать и не завершается, потребляя память и ресурсы. Причины: ожидание на заблокированном канале, бесконечный цикл без условия выхода, забытые фоновые задачи. Предотвращение: использовать `context` для отмены, закрывать каналы после завершения работы, использовать таймауты, применять `select` с `case <-ctx.Done()`, тестировать с помощью детекторов утечек (например, `goleak`).

32. Что произойдет если `main` функция завершится раньше горутин?

Программа немедленно завершится вместе со всеми запущенными горутин. Горутин не успеют завершить свою работу. Для ожидания завершения горутин используется: `sync.WaitGroup`, каналы для синхронизации, или `context` для управления жизненным циклом. `Main` функция должна явно ожидать завершения всех важных горутин.

33. Как ограничить количество одновременно работающих горутин?

Использовать семафор через буферизованный канал: `sem := make(chan struct{}, N)`. Перед запуском задачи `sem <- struct{}{}`, после завершения `<-sem`. Альтернативы: worker pool паттерн (N горутин-воркеров обрабатывают задачи из канала), пакеты golang.org/x/sync/semaphore или `errgroup.WithContext` с ограничением. Это предотвращает исчерпание ресурсов при массовом создании горутин.

34. Что такое `GOMAXPROCS` и как это влияет на производительность?

`GOMAXPROCS` задает максимальное количество системных потоков (M), которые могут одновременно выполнять Go код. По умолчанию равно количеству CPU ядер (`runtime.NumCPU()`). Изменяется через `runtime.GOMAXPROCS(n)` или переменную окружения `GOMAXPROCS`. Увеличение не всегда улучшает производительность — зависит от типа задачи (CPU-bound vs I/O-bound). Для I/O-bound задач значение может быть выше количества ядер.

35. Можно ли получить ID горутин и стоит ли это делать?

Технически можно через рефлексии или парсинг `stack trace`, но это антипаттерн. Go намеренно не предоставляет API для получения goroutine ID. Причины: поощряет правильные паттерны синхронизации через каналы и `context`, избегает thread-local storage антипаттернов, упрощает reasoning о коде. Если нужно отслеживать выполнение — используйте `context.WithValue` для передачи идентификаторов запросов.

36. Как безопасно передавать данные между горутин?

Каналы — идиоматичный способ: `ch := make(chan Type)`, отправка `ch <- value`, получение `value := <-ch`. Для shared state используйте мьютексы (`sync.Mutex`, `sync.RWMutex`). Следуйте принципу "Don't communicate by sharing memory; share memory by communicating". Альтернативы: `sync.Map` для concurrent map, `atomic` для простых значений, immutable структуры (передача копий).

Каналы (Channels)

31. Как работают каналы в Go? Буферизованные vs небуферизованные

Канал — типобезопасная очередь для коммуникации между горутинами. Небуферизованный канал: отправка блокируется до получения, получение блокируется до отправки. Синхронная коммуникация. Буферизованный канал: имеет очередь фиксированного размера, отправка блокируется только при заполнении буфера, получение — при пустом буфере. Асинхронная коммуникация до заполнения буфера.

32. Что произойдет при записи в закрытый канал?

Вызовет панику `panic: send on closed channel`. Закрытый канал нельзя использовать для отправки. Перед отправкой нужно проверять, не закрыт ли канал, или использовать `recover()` для обработки паники. Обычно закрывает канал отправитель (producer), а получатели (consumers) проверяют второе значение `val, ok := <-ch`.

33. Что произойдет при чтении из закрытого канала?

Чтение из закрытого канала возвращает zero value типа и `false` как второе значение: `val, ok := <-ch`, где `ok == false` означает закрытие канала. Можно читать бесконечно — всегда будет zero value. Это позволяет получателям определить, что канал закрыт, и завершить работу.

34. Как правильно закрывать каналы?

Закрывать должен отправитель (producer), а не получатель. Закрывать только один раз — повторное закрытие вызывает панику. Использовать `defer close(ch)` для гарантии закрытия. Получатели проверяют `val, ok := <-ch` и завершаются при `ok == false`. Для множественных отправителей использовать дополнительный канал или `sync.Once`.

35. Что такое `select` и как он работает?

`select` — конструкция для ожидания нескольких каналов одновременно. Выполняет один из готовых `case`, выбирая случайно при нескольких готовых. Блокируется, если все каналы не готовы (если нет `default`). Используется для: мультиплексирования каналов, таймаутов, неблокирующих операций с `default`.

36. Что такое `default` в `select` и когда его использовать?

`default` в `select` выполняется немедленно, если ни один канал не готов. Делает операцию неблокирующей. Используется для: проверки готовности канала без блокировки, реализации таймеров, обработки приоритетных каналов. Без `default` `select` блокируется до готовности хотя бы одного канала.

37. Как использовать `select` для таймаутов?

Использовать `time.After(duration)` в одном из `case`: `select { case val := <-ch: ... case <-time.After(5*time.Second): ... }`. Или `context.WithTimeout` с `case <-ctx.Done()`. Таймаут срабатывает, если операция не завершилась за указанное время. `time.After` создает новый таймер каждый раз, для повторного использования лучше `time.NewTimer`.

38. Что такое `nil` канал и как он ведет себя в `select`?

`nil` канал никогда не готов — операции с ним всегда блокируются. В `select` `case` с `nil` каналом никогда не выберется. Используется для динамического включения/отключения каналов в `select`: установить канал в `nil` для отключения, восстановить для включения. Позволяет условно участвовать в `select`.

39. Как реализовать паттерн fan-in и fan-out с помощью каналов?

Fan-out: одна горутина читает из канала и распределяет работу между несколькими worker-горутинками. Fan-in: несколько горутин отправляют в один канал, одна горутина собирает результаты. Реализация: для fan-out запустить N workers, читающих из общего канала. Для fan-in использовать `select` для чтения из нескольких каналов в один, или закрыть входные каналы и собрать через `range`.

Интерфейсы

40. Как работают интерфейсы в Go? Что такое duck typing?

Интерфейс в Go — набор методов. Тип реализует интерфейс неявно, если содержит все методы интерфейса (structural typing, "duck typing"). Реализация проверяется во время компиляции. Интерфейсное значение содержит указатель на данные и таблицу методов. Duck typing: "если что-то ходит как утка и крикает как утка, то это утка" — тип соответствует интерфейсу по поведению, а не по объявлению.

41. Что такое пустой интерфейс `interface{}` и когда его использовать?

`interface{}` — интерфейс без методов, реализуется любым типом. Используется для: работы с неизвестными типами, generic-подобной функциональности (до Go 1.18), сериализации, рефлексии. Требует type assertion для использования. В Go 1.18+ предпочтительнее использовать `any` (алиас для `interface{}`) или дженерики.

42. В чем разница между `any` и `interface{}`?

`any` — это алиас для `interface{}`, введенный в Go 1.18. Функционально идентичны. `any` более читаемый и явно выражает намерение работать с любым типом. Рекомендуется использовать `any` в новом коде для лучшей читаемости. Оба компилируются одинаково.

43. Как работает type assertion и type switch?

Type assertion: `val, ok := x.(Type)` — проверяет, является ли `x` типом `Type`, возвращает значение и булев флаг. `val := x.(Type)` — паникует при несоответствии. Type switch: `switch x.(type) { case int: ... case string: ... }` — переключается по типу значения. Используется для работы с `interface{}` / `any`, определения конкретного типа в runtime.

44. Что такое embedding интерфейсов?

Embedding — встраивание интерфейса в другой интерфейс. Все методы встроенного интерфейса становятся частью внешнего. Позволяет комбинировать интерфейсы: `type ReadWriter interface { Reader; Writer }`. Тип, реализующий `ReadWriter`, должен реализовать методы и `Reader`, и `Writer`. Упрощает композицию интерфейсов.

45. Можно ли реализовать интерфейс для типа из другого пакета?

Да, можно. Интерфейсы в Go используют структурную типизацию — тип автоматически реализует интерфейс, если содержит нужные методы, независимо от пакета. Это позволяет расширять функциональность чужих типов без модификации исходного кода. Например, можно сделать тип из стандартной библиотеки реализацией кастомного интерфейса.

46. Что такое `io.Reader` и `io.Writer`?

`io.Reader` — интерфейс для чтения: `Read([]byte) (int, error)`. `io.Writer` — интерфейс для записи: `Write([]byte) (int, error)`. Фундаментальные интерфейсы для работы с потоками данных. Реализованы файлами, сетью, буферами, строками. Позволяют писать универсальный код, работающий с любым источником/приемником данных. Основа многих абстракций в Go.

47. Как работает сборщик мусора (GC) в Go?

GC в Go — concurrent mark-and-sweep сборщик. Работает параллельно с программой (не останавливает выполнение полностью). Использует триколорный алгоритм маркировки. Фазы: маркировка достижимых объектов (mark), очистка недостижимых (sweep). Настраивается через `GOGC` (target percentage, по умолчанию 100%). Цель: минимизировать паузы при сохранении низкого overhead.

Go 1.26 — Green Tea GC (по умолчанию). С Go 1.26 вместо старого маркера включён **Green Tea** (в Go 1.25 был экспериментом). Модель GC не меняется: тот же concurrent mark-and-sweep и триколорная абстракция, новых write barrier нет. Меняется только **как** проходит фаза mark: маркировка и сканирование **мелких объектов** идёт с учётом locality — объекты обходятся **по span'ам** (кускам кучи) последовательно, а не через случайный pointer chasing по всей памяти. Это даёт меньше cache miss'ов и лучше масштабируется на несколько ядер. На GC-нагруженных программах ожидается **–10–40% overhead** от сборки; на новых amd64 (Intel Ice Lake, AMD Zen 4+) ещё ~10% за счёт vector-инструкций при сканировании. Количество аллокаций Green Tea **не уменьшает** — только ускоряет сбор. Отключить: `GOEXPERIMENT=nogreenteagc` при сборке (opt-out планируют убрать в Go 1.27).

48. Что такое триколорный маркировочный алгоритм?

Триколорный алгоритм — метод маркировки объектов для GC. Цвета: белый (непосещенный), серый (посещенный, но потомки не проверены), черный (посещенный со всеми потомками). Инвариант: нет прямых ссылок от черного к белому. Позволяет выполнять GC параллельно с программой, так как новые объекты помечаются серым и не конфликтуют с маркировкой.

49. Как работает `runtime.GC()` и когда его вызывать?

`runtime.GC()` принудительно запускает полный цикл сборки мусора. Блокирует выполнение до завершения. Обычно не нужно вызывать вручную — GC работает автоматически. Используется для: бенчмарков (стабильность измерений), тестов производительности, редких случаев когда нужна гарантированная очистка перед критической операцией. В production обычно не требуется.

50. Что такое escape analysis и как он работает?

Escape analysis — анализ компилятора, определяющий, может ли переменная "убежать" из стека в кучу. Если переменная используется после возврата функции или передается в другую горутину, она размещается в куче. Цель: максимизировать размещение в стеке (быстрее). Можно проверить через `go build -gcflags=-m`. Влияет на производительность и GC pressure.

51. Когда переменная попадает в кучу (heap), а когда остается в стеке?

В кучу: если используется после возврата функции, передается в другую горутину, размер неизвестен на этапе компиляции, слишком большая для стека, адрес передается в функцию, которая может сохранить его. В стеке: локальные переменные, используемые только внутри функции, размер известен на этапе компиляции. Компилятор старается максимизировать размещение в стеке.

52. Что такое выравнивание (alignment) структур и почему это важно?

Выравнивание — размещение полей структуры по границам, кратным размеру типа (для эффективного доступа к памяти). Компилятор добавляет padding (пустые байты) между полями. Важно для: производительности (невыверенный доступ медленнее), размера структуры (порядок полей влияет на размер), совместимости с C структурами. Можно проверить через `unsafe.Offsetof`, `unsafe.Sizeof`.

53. Как оптимизировать использование памяти в Go?

Стратегии: минимизировать аллокации (переиспользование буферов, object pooling), уменьшить размер структур (оптимизация порядка полей), использовать слайсы с предварительной capacity, избегать утечек (закрывать каналы, отменять контексты), профилировать через `pprof`, использовать `sync.Pool` для временных объектов, минимизировать escape в кучу.

54. Что такое `runtime.SetFinalizer` и когда его использовать?

`runtime.SetFinalizer(obj, func)` устанавливает финализатор, вызываемый перед сборкой объекта GC. Используется редко, только для: освобождения внешних ресурсов (С память, файловые дескрипторы), логирования, отладки. Не гарантирует вызов (программа может завершиться раньше), не должен блокироваться, не должен ссылаться на объект. Предпочтительнее явное управление ресурсами (`defer`, `close()`).

Ошибки и обработка ошибок

55. Как обрабатываются ошибки в Go? Почему нет исключений?

Ошибки в Go — обычные значения, возвращаемые как последний параметр функции. Явная обработка через проверку `if err != nil`. Нет исключений для: явности (видно все места обработки ошибок), производительности (нет overhead try/catch), простоты (нет сложных стеков вызовов). Паника (`panic`) используется только для критических ошибок, которые нельзя обработать. Обычные ошибки — через возвращаемое значение.

56. Что такое `error` интерфейс?

`error` — встроенный интерфейс: `type error interface { Error() string }`. Любой тип с методом `Error() string` является ошибкой. Стандартные реализации: `errors.New()`, `fmt.Errorf()`. Позволяет создавать кастомные типы ошибок с дополнительной информацией. Проверка через `err != nil`, сравнение через `errors.Is()`, `errors.As()`.

57. Как правильно оборачивать ошибки? `fmt.Errorf` vs `errors.Wrap`

`fmt.Errorf("описание: %w", err)` — оборачивает ошибку с контекстом, сохраняя исходную через `%w` (Go 1.13+). `errors.Wrap(err, "описание")` — из пакета `github.com/pkg/errors`, добавляет stack trace. Рекомендуется `fmt.Errorf` с `%w` для совместимости со стандартной библиотекой. Проверка обернутых ошибок через `errors.Is()`, `errors.As()`. Добавлять контекст на каждом уровне абстракции.

58. Что такое `errors.Is` и `errors.As`?

`errors.Is(err, target)` — проверяет, является ли ошибка или любая обернутая ошибка равной `target`. Работает с обернутыми ошибками через `%w`. `errors.As(err, target)` — проверяет, можно ли привести ошибку к типу `target`, и присваивает значение. Работает с обернутыми ошибками и `type assertion`. Оба используются для проверки обернутых ошибок вместо прямого сравнения.

59. Как создать кастомный тип ошибки?

Создать тип с методом `Error() string`: `type MyError struct { Code int; Msg string }` с `func (e MyError) Error() string { return e.Msg }`. Можно добавить методы для проверки: `func (e MyError) Is(target error) bool` для `errors.Is()`, `func (e MyError) As(target interface{}) bool` для `errors.As()`. Использовать для ошибок с дополнительной информацией (коды, контекст, recoverable состояния).

Тестирование

60. Как писать unit-тесты в Go?

Тесты — функции вида `func TestXxx(t *testing.T)` в файлах `*_test.go`. Запуск: `go test`. Использовать `t.Error()`, `t.Fatal()` для ошибок. `t.Fatal()` останавливает тест, `t.Error()` продолжает. Имя теста должно начинаться с `Test`. Можно использовать подтесты через `t.Run()`. Тесты выполняются параллельно по умолчанию (если не использовать флаги).

61. Что такое table-driven tests?

Table-driven tests — паттерн тестирования с таблицей тест-кейсов. Массив структур с входными данными и ожидаемым результатом, цикл по таблице с `t.Run()` для каждого кейса. Преимущества: компактность, легко добавлять кейсы, единообразная структура. Стандартный подход в Go для тестирования функций с множественными входными данными.

62. Как использовать `t.Helper()`?

`t.Helper()` помечает функцию как test helper. Stack trace при ошибке будет указывать на вызывающий код, а не на helper. Используется в вспомогательных функциях тестирования для правильного отображения места ошибки. Вызывать в начале helper-функции. Улучшает читаемость отчетов об ошибках.

63. Что такое `testify` и для чего он нужен?

`testify` — библиотека расширений для тестирования. Компоненты: `assert` (проверки с понятными сообщениями), `require` (проверки с остановкой теста), `suite` (организация тестов в наборы), `mock` (мокирование). Упрощает написание тестов, улучшает читаемость. Не обязательна, но популярна в сообществе Go.

64. Как писать benchmark тесты?

Benchmark — функции вида `func BenchmarkXxx(b *testing.B)`. Запуск: `go test -bench=.`. Использовать `b.N` для количества итераций. Запускать несколько раз для стабильности: `-benchtime`, `-count`. Избегать побочных эффектов между итерациями. Использовать `b.ResetTimer()`, `b.StopTimer()` для исключения setup из измерений. Сравнивать через `benchstat` или `benchcmp`.

65. Как измерять покрытие кода тестами?

Запуск с флагом: `go test -cover`. Детальный отчет: `go test -coverprofile=coverage.out`, затем `go tool cover -html=coverage.out`. Показывает процент покрытых строк. Цель: высокое покрытие критических путей, не обязательно 100%. Использовать для выявления непротестированных участков кода. Интегрировать в CI/CD.

66. Что такое `httptest` и как тестировать HTTP handlers?

`httptest` — пакет для тестирования HTTP кода. `httptest.NewRequest()` создает `*http.Request`, `httptest.NewRecorder()` создает `http.ResponseWriter` для записи ответа. Тестировать handlers изолированно без реального сервера. Проверять статус код, заголовки, тело ответа. Для интеграционных тестов использовать `httptest.NewServer()` для создания тестового HTTP сервера.

Пакеты и модули

67. Как работает система модулей в Go (go.mod)?

Модули — единица зависимостей с версионированием. `go.mod` определяет модуль: путь модуля, версия Go, зависимости с версиями. Управление: `go mod init`, `go mod tidy` (синхронизация), `go mod download`, `go get` (добавление зависимостей). Версионирование через семантические версии (v1.2.3) или псевдо-версии для коммитов. Заменяет старую систему `GOPATH`.

68. В чем разница между `GOPATH` и модулями?

`GOPATH` — старая система: все проекты в одной директории, зависимости в `$GOPATH/src`, нет версионирования зависимостей. Модули (Go 1.11+): каждый проект — отдельный модуль, зависимости в `go.mod`, семантическое версионирование, работа вне `GOPATH`. Модули — современный стандарт, `GOPATH` устарел для новых проектов. Модули позволяют работать в любой директории.

69. Что такое `vendor` директория?

`vendor/` — директория с копиями зависимостей внутри проекта. Создается через `go mod vendor`. При сборке Go использует зависимости из `vendor/` вместо модульного кэша. Используется для: гарантии версий зависимостей, офлайн-сборки, контроля зависимостей. Обычно добавляется в VCS для воспроизводимых сборок. Команда `go build -mod=vendor` принудительно использует `vendor`.

70. Как работает `go.sum` файл?

`go.sum` содержит криптографические хеши зависимостей для проверки целостности. Генерируется автоматически при `go mod download`, `go build`. Содержит хеши для всех версий зависимостей (прямых и транзитивных). Используется для: проверки неизменности зависимостей, обнаружения подмены, воспроизводимых сборок. Должен коммититься в VCS вместе с `go.mod`.

71. Что такое `internal` пакет?

`internal/` — специальная директория, пакеты внутри доступны только родительскому модулю и его подмодулям. Защищает внутренние API от внешнего использования. Пакет `a/b/internal/c` доступен только пакетам внутри `a/b/`, но не `a/d/`. Используется для: скрытия реализации, предотвращения зависимости от внутренних API, организации частных компонентов модуля.

Практические задачи и алгоритмы

72. Реализовать LRU кэш

LRU (Least Recently Used) кэш — структура данных с ограниченной емкостью, удаляющая наименее используемые элементы. Реализация: комбинация hash map ($O(1)$ доступ) и двусвязного списка ($O(1)$ обновление порядка). При доступе элемент перемещается в голову списка. При переполнении удаляется хвост списка. Операции Get/Put за $O(1)$.

```
type LRUCache struct {
    capacity int
    cache    map[int]*Node // ключ → узел списка,  $O(1)$  поиск
    head     *Node      // самый «свежий» элемент
    tail     *Node      // самый «старый», его выкидываем при переполнении
}

type Node struct {
    key, value int
    prev, next *Node // двусвязный список для порядка использования
}

func (lru *LRUCache) Get(key int) int {
    if node, ok := lru.cache[key]; ok {
        lru.moveToHead(node) // прочитали — элемент стал недавно использованным
        return node.value
    }
    return -1 // ключ не найден
}

func (lru *LRUCache) Put(key, value int) {
    if node, ok := lru.cache[key]; ok {
        // ключ уже есть — обновляем значение и поднимаем в голову
        node.value = value
        lru.moveToHead(node)
    } else {
        if len(lru.cache) >= lru.capacity {
            lru.removeTail() // места нет — удаляем LRU-элемент с хвоста
        }
        newNode := &Node{key: key, value: value}
        lru.cache[key] = newNode
        lru.addToHead(newNode) // новый элемент — самый свежий
    }
}

// moveToHead, removeTail, addToHead — вспомогательные методы работы со списком
```

73. Реализовать thread-safe счетчик

Thread-safe счетчик с использованием `sync.Mutex` или `atomic` операций. Для простых операций `atomic` эффективнее, для сложной логики — мьютекс.

```
// Вариант 1: atomic — быстрее для простого инкремента, без блокировок
type Counter struct {
    value int64
}

func (c *Counter) Increment() {
    atomic.AddInt64(&c.value, 1) // атомарно +1 на уровне CPU
}

func (c *Counter) Value() int64 {
    return atomic.LoadInt64(&c.value) // атомарное чтение
}

// Вариант 2: mutex — если логика сложнее одной операции
type Counter struct {
    mu sync.Mutex
    value int64
}

func (c *Counter) Increment() {
    c.mu.Lock()
    defer c.mu.Unlock() // разблокируем при выходе из функции
    c.value++
}

func (c *Counter) Value() int64 {
    c.mu.Lock()
    defer c.mu.Unlock()
    return c.value
}
```

74. Реализовать пул горутин (worker pool)

Worker pool — фиксированное количество worker-горутин обрабатывает задачи из канала. Контроль конкурентности, переиспользование горутин.

```

func workerPool(jobs <-chan Job, results chan<- Result, numWorkers int) {
    var wg sync.WaitGroup

    // поднимаем фиксированное число воркеров – не по одной горутине на задачу
    for i := 0; i < numWorkers; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for job := range jobs { // читаем, пока jobs не закроют
                results <- process(job)
            }
        }()
    }

    wg.Wait() // ждём, пока все воркеры обработают очередь
    close(results) // после этого в results больше ничего не придёт
}

```

75. Реализовать rate limiter

Rate limiter ограничивает количество запросов за период времени. Реализации: token bucket, sliding window, fixed window.

```

type RateLimiter struct {
    tokens chan struct{} // «жетоны»: забрал – разрешил запрос
    ticker *time.Ticker    // периодически пополняет bucket
}

func NewRateLimiter(rate int, per time.Duration) *RateLimiter {
    rl := &RateLimiter{
        tokens: make(chan struct{}, rate), // буфер = максимум жетонов
        ticker: time.NewTicker(per / time.Duration(rate)),
    }

    // фоновая горутина: равномерно кидает жетоны в bucket
    go func() {
        for range rl.ticker.C {
            select {
            case rl.tokens <- struct{}{}: // положили жетон
            default: // bucket полон – лишний жетон выбрасываем
            }
        }
    }()

    return rl
}

func (rl *RateLimiter) Allow() bool {
    select {
    case <-rl.tokens: // есть жетон – запрос разрешён
        return true
    default: // жетонов нет – лимит исчерпан
        return false
    }
}

```

76. Реализовать простой HTTP сервер с middleware

HTTP сервер с цепочкой middleware для логирования, аутентификации, обработки ошибок.

```

// Middleware оборачивает handler и может сделать что-то до/после него
type Middleware func(http.HandlerFunc) http.HandlerFunc

func chain(middlewares ...Middleware) Middleware {
    return func(next http.HandlerFunc) http.HandlerFunc {
        // оборачиваем с конца: первый в списке – внешний слой
        for i := len(middlewares) - 1; i >= 0; i-- {
            next = middlewares[i](next)
        }
        return next
    }
}

func logging(next http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next(w, r) // вызываем следующий handler в цепочке
        log.Printf("%s %s %v", r.Method, r.URL.Path, time.Since(start))
    }
}

func main() {
    handler := chain(logging)(myHandler) // logging → myHandler
    http.HandleFunc("/", handler)
    http.ListenAndServe(":8080", nil)
}

```

77. Реализовать producer-consumer паттерн

Producer генерирует данные, consumer обрабатывает. Синхронизация через каналы.

```
func producer(items chan<- int) {
    for i := 0; i < 10; i++ {
        items <- i // кладём задачи в канал
    }
    close(items) // сигнал consumer'у: данных больше не будет
}

func consumer(items <-chan int, done chan<- bool) {
    for item := range items { // выходим, когда producer закроет канал
        process(item)
    }
    done <- true // сообщаем main, что всё обработано
}

func main() {
    items := make(chan int)
    done := make(chan bool)

    go producer(items)
    go consumer(items, done)

    <-done // main ждёт завершения consumer
}
```

78. Реализовать merge нескольких каналов

Объединение нескольких каналов в один с использованием `select` и горутин.

```
func merge(channels ...<-chan int) <-chan int {
    out := make(chan int)
    var wg sync.WaitGroup

    // на каждый входной канал — своя горутина-перекачка
    for _, ch := range channels {
        wg.Add(1)
        go func(c <-chan int) {
            defer wg.Done()
            for v := range c {
                out <- v // пересылаем значения в общий канал
            }
        }(ch) // ch копируем в аргумент — иначе замыкание на последний ch
    }

    // закрываем out только когда все входные каналы исчерпаны
    go func() {
        wg.Wait()
        close(out)
    }()

    return out
}
```

79. Реализовать timeout для операции

Таймаут для операции через `context.WithTimeout` или `select` с `time.After`.

```
func withTimeout(fn func() error, timeout time.Duration) error {
    ctx, cancel := context.WithTimeout(context.Background(), timeout)
    defer cancel() // освобождаем таймер, даже если fn уже вернулась

    done := make(chan error, 1) // буфер 1 — горутина не зависнет после return
    go func() {
        done <- fn() // тяжёлую работу делаем в отдельной горутине
    }()

    select {
    case err := <-done:
        return err // успели до дедлайна
    case <-ctx.Done():
        return ctx.Err() // context.DeadlineExceeded
    }
}
```

80. Реализовать graceful shutdown сервера

Graceful shutdown — корректное завершение с ожиданием завершения текущих запросов.

```

func main() {
    server := &http.Server{Addr: ":8080"}

    // сервер в фоне – main свободен ловить сигналы ОС
    go func() {
        if err := server.ListenAndServe(); err != nil && err != http.ErrServerClosed {
            log.Fatal(err)
        }
    }()

    quit := make(chan os.Signal, 1)
    signal.Notify(quit, os.Interrupt, syscall.SIGTERM) // Ctrl+C или kill
    <-quit // блокируемся до сигнала остановки

    // даём in-flight запросам до 5 секунд на завершение
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    if err := server.Shutdown(ctx); err != nil {
        log.Fatal("Server forced to shutdown:", err)
    }
}

```

81. Как спроектировать высоконагруженную систему на Go?

Принципы: горизонтальное масштабирование (stateless сервисы), асинхронная обработка (каналы, очереди), кэширование (in-memory, Redis), connection pooling, graceful degradation, circuit breaker, rate limiting, мониторинг и метрики. Использовать горутин для конкурентной обработки, избегать блокирующих операций, оптимизировать аллокации памяти, использовать профилирование для узких мест.

82. Какие паттерны проектирования часто используются в Go?

Популярные паттерны: Dependency Injection (через интерфейсы), Factory, Builder, Strategy (через интерфейсы), Observer (через каналы), Pipeline (через каналы), Worker Pool, Context для cancellation/timeout. Go предпочитает композицию наследованию, интерфейсы для абстракций, каналы для коммуникации. Многие классические паттерны упрощаются благодаря горутинам и каналам.

83. Как обеспечить отказоустойчивость микросервисов?

Стратегии: health checks, circuit breaker (предотвращение каскадных отказов), retry с exponential backoff, timeout на всех внешних вызовах, fallback механизмы, graceful degradation, мониторинг и алертинг, изоляция отказов (bulkhead pattern), rate limiting для защиты от перегрузки. Использовать `context` для таймаутов, библиотеки типа `hystrix-go` или `gobreaker` для circuit breaker.

84. Как реализовать circuit breaker на Go?

Circuit breaker предотвращает каскадные отказы, блокируя вызовы при превышении порога ошибок. Состояния: Closed (норма), Open (блокировка), Half-Open (тестирование восстановления). Реализация: счетчик ошибок, таймер для перехода в Half-Open, логика переключения состояний. Библиотеки: `sony/gobreaker`, `afex/hystrix-go`. Можно реализовать на основе счетчиков и мьютексов с проверкой состояния перед вызовом.

85. Как организовать graceful shutdown в микросервисе?

Graceful shutdown: перехват сигналов (SIGTERM, SIGINT), остановка приема новых запросов, ожидание завершения текущих запросов с таймаутом, закрытие соединений (БД, очереди), освобождение ресурсов. Использовать `context` для отмены операций, `sync.WaitGroup` для отслеживания горутин, `http.Server.Shutdown()` для HTTP серверов. Критично для zero-downtime деплоев и корректной обработки in-flight запросов.

Работа с базами данных

86. Как работать с SQL базами данных в Go?

Использовать пакет `database/sql` — стандартный интерфейс для работы с SQL БД. Драйверы: `pq` (PostgreSQL), `go-sql-driver/mysql`, `mattn/go-sqlite3`. Основные операции: `db.Open()`, `db.Query()`, `db.Exec()`, `db.Prepare()`. Всегда закрывать `rows` и `stmt`, обрабатывать ошибки, использовать `prepared statements` для безопасности и производительности. `Connection pooling` встроен в `database/sql`.

87. Что такое `database/sql` и как он работает?

`database/sql` — стандартный интерфейс для работы с SQL БД, абстракция над драйверами. Управляет `connection pool` автоматически: создает, переиспользует, закрывает соединения. Настройки: `SetMaxOpenConns()`, `SetMaxIdleConns()`, `SetConnMaxLifetime()`. Предоставляет типизированные методы: `Query()` для `SELECT`, `Exec()` для `INSERT/UPDATE/DELETE`, `Prepare()` для `prepared statements`. Абстрагирует различия между БД.

88. Как работают `connection pools` в Go?

`Connection pool` управляется `database/sql` автоматически. Настройки: `MaxOpenConns` (максимум открытых), `MaxIdleConns` (максимум простаивающих), `ConnMaxLifetime` (время жизни соединения). При запросе: берется соединение из пула, используется, возвращается. При нехватке создается новое (до `MaxOpenConns`). Простаивающие соединения закрываются после `ConnMaxLifetime`. Важно настроить под нагрузку для оптимальной производительности.

89. Как использовать транзакции?

Транзакции через `db.Begin()`, возвращает `*sql.Tx`. Методы: `tx.Exec()`, `tx.Query()`, `tx.Commit()`, `tx.Rollback()`. Всегда использовать `defer tx.Rollback()` для отката при ошибке, вызывать `tx.Commit()` при успехе. Транзакции изолируют операции, обеспечивают ACID свойства. Использовать для атомарных операций, состоящих из нескольких запросов. Не забывать закрывать транзакции.

90. Какие ORM используются в Go? (GORM, `sqlx` и т.д.)

Популярные библиотеки: GORM (полнофункциональный ORM с миграциями, ассоциациями), `sqlx` (расширение `database/sql` с удобной работой со структурами), `ent` (Facebook, code generation), `sqlboiler` (code generation). GORM — самый популярный, но тяжелее. `sqlx` — легковесный, ближе к SQL. Выбор зависит от сложности проекта: простые запросы — `sqlx`, сложные модели — GORM или `ent`.

HTTP и веб-разработка

91. Как работает `net/http` пакет?

`net/http` — стандартный пакет для HTTP клиентов и серверов. Сервер: `http.ListenAndServe()`, `http.HandleFunc()`, `http.Handler` интерфейс. Клиент: `http.Get()`, `http.Post()`, `http.Client` для настройки. Обрабатывает HTTP протокол, управляет соединениями, парсит заголовки. `http.Request` содержит метод, URL, заголовки, тело. `http.ResponseWriter` для записи ответа. Основа веб-разработки в Go.

92. В чем разница между `http.Handle` и `http.HandleFunc`?

`http.Handle(pattern, handler)` принимает `http.Handler` интерфейс (метод `ServeHTTP`). `http.HandleFunc(pattern, handler)` принимает функцию `func(http.ResponseWriter, *http.Request)`. `HandleFunc` — удобная обертка, конвертирующая функцию в `http.Handler`. Функционально эквивалентны, выбор зависит от стиля: функции — `HandleFunc`, структуры с состоянием — `Handle` с кастомным `Handler`.

93. Как создать middleware для HTTP handlers?

Middleware — функция, оборачивающая handler для добавления функциональности (логирование, аутентификация, CORS). Паттерн: `func(http.HandlerFunc) http.HandlerFunc` или `func(http.Handler) http.Handler`. Цепочка middleware через композицию функций. Примеры: логирование запросов, проверка авторизации, добавление заголовков, rate limiting. Можно использовать библиотеки типа `gorilla/mux` для удобной композиции.

94. Как работает `context` в HTTP запросах?

Каждый HTTP запрос имеет `context`, доступный через `r.Context()`. Автоматически отменяется при закрытии соединения клиентом. Используется для: передачи request-scoped данных (request ID, user info), таймаутов, отмены долгих операций. Передавать `ctx` во все асинхронные операции. `http.Request.WithContext()` создает новый запрос с контекстом. Критично для корректной обработки отмены запросов.

95. Как реализовать graceful shutdown HTTP сервера?

Использовать `http.Server.Shutdown(ctx)` вместо прямого закрытия. Процесс: перехватить сигнал (SIGTERM, SIGINT), вызвать `server.Shutdown()` с таймаутом, сервер перестанет принимать новые запросы и дождется завершения текущих (до таймаута). После таймаута принудительно закрывает соединения. Позволяет обработать in-flight запросы, критично для production без потери данных.

Производительность и профилирование

96. Какие инструменты для профилирования есть в Go?

Основные инструменты: `pprof` (CPU, память, горютины, блокировки), `go tool trace` (детальная трассировка выполнения), `go test -bench` (бенчмарки), `go test -cpuprofile`, `go test -memprofile`. Встроенные в `runtime`, не требуют внешних инструментов. Интеграция через `net/http/pprof` для live профилирования работающих приложений. Стандартные инструменты покрывают большинство случаев.

97. Как использовать `pprof`?

Импортировать `"net/http/pprof"` и запустить HTTP сервер. Доступ через HTTP: `/debug/pprof/` (список профилей), `/debug/pprof/heap` (память), `/debug/pprof/profile` (CPU). Анализ: `go tool pprof http://localhost:6060/debug/pprof/profile`, затем команды `top`, `list`, `web`. Или сохранить профиль и анализировать: `go tool pprof profile.out`. Показывает узкие места, аллокации, использование CPU.

98. Как профилировать CPU и память?

CPU: `go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30` (30 секунд профилирования) или `go test -cpuprofile=cpu.prof`. Память: `go tool pprof http://localhost:6060/debug/pprof/heap` или `go test -memprofile=mem.prof`. Команды анализа: `top` (топ функций), `list FunctionName` (детали функции), `web` (визуализация графа). Показывает где тратится время/память, помогает найти оптимизации.

99. Что такое `go tool trace`?

`go tool trace` — инструмент для детальной трассировки выполнения программы. Показывает: планирование горютин, GC события, системные вызовы, блокировки, сетевые операции. Генерация: `runtime/trace` пакет, `go test -trace=trace.out`. Визуализация: `go tool trace trace.out`, открывает веб-интерфейс. Полезен для анализа конкурентности, понимания поведения планировщика, диагностики проблем производительности.

100. Как оптимизировать производительность Go приложений?

Стратегии: профилирование для выявления узких мест (не угадывать), минимизация аллокаций (переиспользование, `pooling`), оптимизация `hot paths`, использование `sync.Pool` для временных объектов, избегание `escape` в кучу, оптимизация структур (выравнивание), использование `atomic` вместо мьютексов где возможно, `connection pooling` для БД, кэширование, `batch` обработка. Измерять до и после оптимизаций.

Безопасность

101. Какие уязвимости безопасности характерны для Go приложений?

Типичные уязвимости: SQL инъекции (неправильное использование строковых запросов), XSS (неэкранированный вывод), CSRF (отсутствие токенов), утечки секретов (в коде, логах), небезопасная десериализация, path traversal, недостаточная валидация входных данных, использование `unsafe` пакета, race conditions. Go защищает от некоторых уязвимостей (buffer overflows благодаря проверке границ), но требует внимательности разработчика.

102. Как защититься от SQL инъекций?

Всегда использовать prepared statements (`db.Prepare()`, `stmt.Exec()`), никогда не конкатенировать пользовательский ввод в SQL запросы. Параметризованные запросы: `db.Exec("SELECT * FROM users WHERE id = ?", userID)`. Библиотеки типа GORM, sqlx автоматически используют prepared statements. Валидировать и санитизировать входные данные. Использовать whitelist для допустимых значений где возможно.

103. Как безопасно работать с секретами?

Никогда не хранить секреты в коде или VCS. Использовать: переменные окружения, секретные менеджеры (HashiCorp Vault, AWS Secrets Manager), конфигурационные файлы вне VCS, Kubernetes secrets. В production использовать сервисы управления секретами. Не логировать секреты. Ротация секретов. Использовать минимальные права доступа (principle of least privilege). Шифрование секретов в покое и при передаче.

104. Что такое `unsafe` пакет и когда его использовать?

`unsafe` — пакет для низкоуровневых операций, обходящих систему типов Go. Функции: `unsafe.Pointer` (преобразование указателей), `unsafe.Sizeof`, `unsafe.Offsetof`, `unsafe.Alignof`. Использовать только когда: критична производительность, работа с C кодом, оптимизация структур данных, низкоуровневые системные вызовы. Опасен: может привести к панике, нарушению безопасности памяти, несовместимости между версиями Go. Избегать без крайней необходимости.

Сети и сетевые протоколы

91. Что такое модель OSI? Какие у нее уровни?

Модель OSI (Open Systems Interconnection) — теоретическая модель сетевого взаимодействия из 7 уровней: 1) Physical (физический) — передача битов по среде; 2) Data Link (канальный) — передача фреймов между узлами (Ethernet, MAC); 3) Network (сетевой) — маршрутизация пакетов (IP); 4) Transport (транспортный) — доставка данных между процессами (TCP, UDP); 5) Session (сеансовый) — управление сеансами; 6) Presentation (представления) — форматирование данных; 7) Application (прикладной) — пользовательские приложения (HTTP, FTP, SMTP).

92. Что такое модель TCP/IP? Чем отличается от OSI?

TCP/IP — практическая модель из 4 уровней: 1) Link (канальный) — соответствует Physical + Data Link OSI; 2) Internet (сетевой) — маршрутизация (IP, ICMP); 3) Transport (транспортный) — TCP, UDP; 4) Application (прикладной) — HTTP, DNS, FTP. Отличия от OSI: меньше уровней, более практичная, фактически используется в Интернете. OSI — теоретическая, TCP/IP — реализованная основа современного Интернета.

93. В чем разница между TCP и UDP?

TCP (Transmission Control Protocol) — надежный, с установлением соединения, гарантирует доставку и порядок пакетов, контроль перегрузки, медленнее. UDP (User Datagram Protocol) — ненадежный, без соединения, не гарантирует доставку и порядок, быстрее, меньше overhead. TCP для критичных данных (HTTP, FTP, email), UDP для потокового видео, игр, DNS где важна скорость, а потери допустимы.

94. Как работает TCP handshake (трехстороннее рукопожатие)?

Трехэтапное установление соединения: 1) SYN — клиент отправляет SYN пакет с начальным sequence number; 2) SYN-ACK — сервер отвечает SYN-ACK с подтверждением и своим sequence number; 3) ACK — клиент отправляет ACK, подтверждая получение. После этого соединение установлено, начинается передача данных. Завершение через 4-way handshake: FIN, ACK, FIN, ACK.

95. Что такое HTTP и какие у него методы?

HTTP (HyperText Transfer Protocol) — протокол прикладного уровня для передачи данных в веб. Основные методы: GET (получение ресурса), POST (создание ресурса), PUT (полное обновление), PATCH (частичное обновление), DELETE (удаление), HEAD (получение заголовков без тела), OPTIONS (допустимые методы для ресурса). Stateless протокол, работает поверх TCP, порт 80 (HTTP) или 443 (HTTPS).

96. В чем разница между HTTP/1.1, HTTP/2 и HTTP/3?

HTTP/1.1 — текстовый протокол, одно соединение на запрос (или keep-alive), head-of-line blocking. HTTP/2 — бинарный, мультиплексирование (несколько запросов в одном соединении), Server Push, компрессия заголовков (HPACK), приоритизация, но HOL blocking на уровне TCP. HTTP/3 — работает поверх QUIC (UDP), устраняет TCP HOL blocking, быстрое установление соединения, лучше на нестабильных сетях.

97. Что такое HTTPS и как работает SSL/TLS?

HTTPS — HTTP поверх SSL/TLS, обеспечивает шифрование, аутентификацию и целостность данных. TLS Handshake: 1) Client Hello — клиент отправляет поддерживаемые cipher suites; 2) Server Hello — сервер выбирает suite, отправляет сертификат; 3) клиент проверяет сертификат, генерирует session key, шифрует его публичным ключом сервера; 4) сервер расшифровывает session key приватным ключом; 5) начинается зашифрованная передача симметричным ключом.

98. Что такое DNS и как работает разрешение имен?

DNS (Domain Name System) — система преобразования доменных имен в IP-адреса. Процесс: 1) браузер проверяет кеш; 2) запрос к локальному DNS resolver (ISP); 3) если нет в кеше — рекурсивный запрос: root server → TLD server (.com) → authoritative server домена; 4) ответ возвращается и кешируется. DNS использует UDP (порт 53) для запросов, TCP для зонных трансферов. TTL определяет время жизни записи в кеше.

99. Что такое REST API? Какие у него принципы?

REST (Representational State Transfer) — архитектурный стиль для веб-сервисов. Принципы: 1) Stateless — каждый запрос независим, содержит всю необходимую информацию; 2) Client-Server — разделение интересов; 3) Cacheable — ответы должны определять, кешируемы ли они; 4) Uniform Interface — единообразный интерфейс (HTTP методы, URI); 5) Layered System — клиент не знает о промежуточных слоях; 6) Code on Demand (опционально) — сервер может передавать исполняемый код.

100. Что такое WebSocket и чем отличается от HTTP?

WebSocket — протокол полнодуплексной связи поверх TCP, позволяет двустороннюю передачу данных в реальном времени. Отличия от HTTP: постоянное соединение vs запрос-ответ, двунаправленная передача vs однонаправленная, меньше overhead для частых сообщений. Использование: чаты, онлайн игры, live updates. Устанавливается через HTTP Upgrade запрос, затем переключается на WebSocket протокол (порт 80/443).

101. Что такое порты и для чего они нужны?

Порты — числовые идентификаторы (0-65535) для различения приложений/сервисов на одном хосте. Диапазоны: 0-1023 (well-known, системные: HTTP 80, HTTPS 443, SSH 22, DNS 53), 1024-49151 (registered, пользовательские приложения), 49152-65535 (dynamic/private, временные). Порт + IP адрес = socket, уникально идентифицирует соединение. Протокол транспортного уровня (TCP/UDP) использует порты для мультиплексирования.

102. Что такое NAT и для чего он используется?

NAT (Network Address Translation) — технология преобразования IP-адресов, позволяет множеству устройств в частной сети использовать один публичный IP. Типы: Static NAT (1:1), Dynamic NAT (пул публичных IP), PAT/NAPT (Port Address Translation, множество частных на один публичный через разные порты). Решает проблему нехватки IPv4 адресов, обеспечивает базовую безопасность (скрывает внутреннюю структуру сети).

103. Что такое load balancer и какие типы бывают?

Load Balancer — компонент, распределяющий входящий трафик между несколькими серверами для повышения доступности и производительности. Уровни: L4 (Transport) — балансировка по IP/port, быстрая, не смотрит содержимое; L7 (Application) — балансировка по HTTP заголовкам, URL, cookies, медленнее, но гибче. Алгоритмы: Round Robin, Least Connections, IP Hash, Weighted. Примеры: nginx, HAProxy, AWS ELB/ALB, Envoy.

104. Что такое CDN и как работает?

CDN (Content Delivery Network) — распределенная сеть серверов для доставки контента пользователям с ближайшей географической точки. Снижает latency, нагрузку на origin сервер, улучшает доступность. Работа: статический контент кешируется на edge серверах, при запросе DNS направляет на ближайший сервер, при cache miss запрашивается origin. Используется для статики (изображения, CSS, JS), видео стриминга, API ускорения. Примеры: Cloudflare, Akamai, AWS CloudFront.

105. Что такое Cookie и Session? В чем разница?

Cookie — небольшие данные, хранящиеся на клиенте, отправляются с каждым запросом. Параметры: domain, path, expires, secure, httpOnly, sameSite. Session — данные о пользователе, хранящиеся на сервере, идентифицируются по session ID (обычно в cookie). Разница: Cookie на клиенте (ограничение размера 4KB, небезопасны), Session на сервере (больше данных, безопаснее, требует хранилище). Использование: аутентификация, персонализация, отслеживание.

106. Что такое процесс и поток? В чем разница?

Процесс — независимая единица выполнения с собственным адресным пространством, ресурсами (память, дескрипторы файлов), контекстом выполнения. Поток (thread) — легковесная единица выполнения внутри процесса, разделяет адресное пространство и ресурсы процесса, имеет собственный стек и регистры. Различия: процессы изолированы, создание дороже, IPC сложнее; потоки разделяют память, создание дешевле, коммуникация проще, но нужна синхронизация.

107. Как процессы взаимодействуют между собой (IPC)?

IPC (Inter-Process Communication) — механизмы взаимодействия процессов: 1) Pipes (каналы) — однонаправленная передача данных между родственными процессами; 2) Named Pipes (FIFO) — именованные каналы для несвязанных процессов; 3) Message Queues — очереди сообщений; 4) Shared Memory — общая память, самый быстрый, требует синхронизации; 5) Sockets — сетевое взаимодействие локально или удаленно; 6) Signals — асинхронные уведомления; 7) Semaphores — синхронизация доступа к ресурсам.

108. Что такое системный вызов (syscall)?

Системный вызов — интерфейс между пользовательским процессом и ядром ОС для запроса привилегированных операций. Процесс: приложение вызывает библиотечную функцию → переключение контекста из user mode в kernel mode → выполнение в ядре → возврат результата → переключение обратно в user mode. Примеры: open(), read(), write(), fork(), exec(). Syscall медленнее обычных вызовов из-за переключения контекста и проверки безопасности.

109. Что такое виртуальная память и зачем она нужна?

Виртуальная память — абстракция, позволяющая процессам использовать больше памяти, чем физически доступно, и изолировать адресные пространства. MMU (Memory Management Unit) транслирует виртуальные адреса в физические через таблицы страниц. Преимущества: изоляция процессов (безопасность), простое управление памятью, memory overcommitment, поддержка swar. Каждый процесс видит свое адресное пространство начиная с 0, независимо от других.

110. Что такое page fault и когда он возникает?

Page fault — исключение, возникающее при обращении к странице памяти, которой нет в физической памяти. Типы: 1) Minor (soft) — страница в памяти, но не в таблице страниц процесса; 2) Major (hard) — страница на диске (swap), требует I/O; 3) Invalid — обращение к несуществующему адресу (Segmentation Fault). При page fault ядро загружает страницу в память, обновляет таблицу страниц, возобновляет выполнение. Major page faults медленные из-за дисковых операций.

111. Что такое deadlock и какие условия необходимы для его возникновения?

Deadlock (взаимная блокировка) — ситуация, когда процессы/потоки вечно ждут друг друга. Условия (Coffman): 1) Mutual Exclusion — ресурс используется только одним процессом; 2) Hold and Wait — процесс удерживает ресурс и ждет другой; 3) No Preemption — ресурс нельзя отобрать принудительно; 4) Circular Wait — циклическая зависимость ожидания. Предотвращение: порядок захвата ресурсов, timeouts, deadlock detection алгоритмы, избегание hold and wait.

112. Что такое race condition и как с этим бороться?

Race condition — ситуация, когда поведение программы зависит от недетерминированного порядка выполнения потоков при доступе к shared state. Возникает при: множественный доступ к данным, хотя бы одна операция записи, отсутствие синхронизации. Решения: 1) Mutexes (мьютексы) — блокировка доступа; 2) Atomic операции — неделимые операции; 3) Lock-free структуры; 4) Immutable data — неизменяемые данные; 5) Message passing — передача сообщений вместо shared memory.

113. Что такое context switching и почему он дорогой?

Context switching — переключение CPU с одного процесса/потока на другой. Операции: сохранение регистров, program counter, stack pointer текущего процесса; загрузка состояния следующего процесса; переключение таблиц страниц (для процессов). Дорогой из-за: сохранения/восстановления состояния, инвалидации TLB и кешей процессора, переключения в kernel mode. Частое переключение снижает производительность. Потоки дешевле процессов (не переключают адресное пространство).

114. Что такое scheduling и какие алгоритмы планирования существуют?

Scheduling — процесс выбора следующего процесса/потока для выполнения на CPU. Алгоритмы: 1) FCFS (First-Come, First-Served) — очередь, простой, но долгие процессы блокируют короткие; 2) Round Robin — квант времени каждому, справедливый; 3) Priority Scheduling — по приоритету, риск starvation; 4) Shortest Job First — минимизирует среднее время ожидания; 5) Multi-level Queue — разные очереди с разными приоритетами; 6) CFS (Completely Fair Scheduler) — Linux, виртуальное время выполнения.

115. Что такое файловый дескриптор?

Файловый дескриптор (file descriptor) — целое число, идентифицирующее открытый файл или I/O ресурс в процессе. Ядро поддерживает таблицу дескрипторов для каждого процесса. Стандартные: 0 (stdin), 1 (stdout), 2 (stderr). Операции: open() создает, read()/write() используют, close() освобождает. Ограничения: системный лимит (ulimit -n), процессный лимит. Дескриптор — абстракция над файлами, сокетами, pipes, устройствами. При fork() дочерний процесс наследует копии дескрипторов.

116. Что такое zombie и orphan процессы?

Zombie процесс — завершённый процесс, чей exit status еще не прочитан родителем через wait(). Занимает запись в таблице процессов, но не потребляет ресурсов. Исчезает после wait() родителем. Orphan процесс — процесс, чей родитель завершился раньше. Автоматически "усыновляется" init/systemd (PID 1), который вызывает wait() для очистки. Много zombie указывают на проблему в родительском процессе (не вызывает wait()).

117. Что такое signals в Unix/Linux?

Signals — асинхронные уведомления процессу о событиях. Типы: SIGINT (Ctrl+C, прерывание), SIGTERM (вежливое завершение), SIGKILL (принудительное завершение, нельзя обработать), SIGHUP (hangup, перезагрузка конфига), SIGSEGV (segmentation fault), SIGCHLD (дочерний процесс завершился). Обработка: default action, ignore, custom handler (через signal() или sigaction()). Сигналы могут прервать системные вызовы. kill() отправляет сигнал процессу.

118. Что такое fork() и exec()?

fork() — системный вызов, создающий дочерний процесс как копию родительского. Возвращает: 0 в дочернем, PID дочернего в родительском. Копируется адресное пространство (Copy-on-Write), дескрипторы файлов, переменные окружения. exec() — семейство вызовов (execve, execl, execp) заменяет текущий процесс новой программой. Загружает новый код, инициализирует память, сбрасывает состояние, сохраняет PID и дескрипторы. Паттерн: fork() затем exec() для запуска новой программы в дочернем процессе.

119. Что такое Copy-on-Write (COW)?

Copy-on-Write — оптимизация для копирования данных: вместо немедленного копирования создаются ссылки на оригинал, реальное копирование происходит только при модификации. Используется в: fork() — дочерний процесс разделяет страницы памяти с родительским до записи; файловые системы (btrfs, ZFS) — snapshot без копирования данных; strings в некоторых языках. Преимущества: экономия памяти, быстрое копирование, ленивые вычисления. При записи происходит page fault и копирование страницы.

120. Что такое pipe и как работает?

Pipe — однонаправленный канал связи между процессами, обычно родитель-ребенок. Создается pipe(), возвращает два дескриптора: read end и write end. Данные записываются в write end, читаются из read end, буферизуются в ядре. Блокирующие операции: чтение из пустого pipe блокируется до появления данных; запись в полный pipe блокируется. При закрытии write end чтение возвращает EOF; запись в закрытый read end вызывает SIGPIPE. В shell `|` создает pipe между командами.

Базы данных (общие вопросы)

121. Что такое ACID и для чего нужны эти свойства?

ACID — набор свойств транзакций в БД: 1) Atomicity (атомарность) — транзакция выполняется полностью или не выполняется вообще; 2) Consistency (согласованность) — транзакция переводит БД из одного корректного состояния в другое; 3) Isolation (изолированность) — параллельные транзакции не влияют друг на друга; 4) Durability (долговечность) — зафиксированные изменения сохраняются даже при сбое. Гарантирует надежность и корректность данных в многопользовательской среде.

122. Что такое транзакция и зачем она нужна?

Транзакция — последовательность операций с БД, выполняемая как единое целое. Начинается BEGIN/START TRANSACTION, завершается COMMIT (применение изменений) или ROLLBACK (отмена). Используется для: обеспечения целостности данных при множественных связанных операциях (перевод денег: списание + зачисление), изоляции параллельных операций, возможности отката при ошибках. Транзакции гарантируют ACID свойства.

123. Какие уровни изоляции транзакций существуют?

Четыре уровня (от слабого к сильному): 1) Read Uncommitted — видны незафиксированные изменения (dirty read); 2) Read Committed — видны только зафиксированные данные, но возможны non-repeatable read; 3) Repeatable Read — повторное чтение возвращает те же данные, но возможны phantom read; 4) Serializable — полная изоляция, как последовательное выполнение, самый медленный. Проблемы: Dirty Read (чтение незафиксированных), Non-Repeatable Read (данные изменились между чтениями), Phantom Read (новые строки появились/исчезли).

124. Что такое индекс и для чего он нужен?

Индекс — структура данных, ускоряющая поиск по таблице. Хранит отсортированные значения столбцов со ссылками на строки. Преимущества: быстрый поиск $O(\log n)$ вместо $O(n)$, ускоряет WHERE, JOIN, ORDER BY, GROUP BY. Недостатки: занимает место, замедляет INSERT/UPDATE/DELETE (нужно обновлять индекс), требует обслуживания. Типы: B-tree (сбалансированное дерево, универсальный), Hash (точное равенство), Bitmap (низкая кардинальность), Full-text (текстовый поиск).

125. Какие типы индексов существуют?

Основные типы: 1) B-tree — сбалансированное дерево, универсальный, поддерживает $>$, $<$, $=$, BETWEEN, ORDER BY; 2) Hash — быстрый точный поиск ($=$), не поддерживает диапазоны; 3) GiST/GIN (PostgreSQL) — для полнотекстового поиска, массивов, JSON; 4) Bitmap — для низкой кардинальности (пол, статус), комбинирует несколько индексов; 5) Clustered — данные физически отсортированы по индексу (Primary Key); 6) Non-clustered — отдельная структура. По составу: Single-column, Composite (многостолбцовый), Covering (включает все нужные столбцы).

126. Что такое нормализация БД и какие формы существуют?

Нормализация — процесс организации данных для уменьшения избыточности и аномалий. Нормальные формы: 1NF — атомарные значения, нет повторяющихся групп; 2NF — выполнена 1NF, нет частичной зависимости от ключа; 3NF — выполнена 2NF, нет транзитивной зависимости; BCNF — каждый детерминант — суперключ; 4NF, 5NF — более строгие. Обычно достаточно 3NF. Денормализация — намеренное добавление избыточности для производительности (кеширование агрегатов, дублирование данных).

127. Что такое JOIN и какие типы бывают?

JOIN — операция объединения строк из двух или более таблиц по условию. Типы: 1) INNER JOIN — возвращает только совпадающие строки из обеих таблиц; 2) LEFT JOIN (LEFT OUTER) — все строки из левой таблицы + совпадения из правой (NULL если нет); 3) RIGHT JOIN — все из правой + совпадения из левой; 4) FULL OUTER JOIN — все строки из обеих таблиц; 5) CROSS JOIN — декартово произведение, каждая строка с каждой; 6) SELF JOIN — таблица соединяется сама с собой.

128. Что такое первичный ключ (Primary Key) и внешний ключ (Foreign Key)?

Primary Key — уникальный идентификатор строки в таблице, не может быть NULL, только один на таблицу. Обеспечивает уникальность и идентификацию записей, автоматически создает индекс. Foreign Key — столбец, ссылающийся на Primary Key другой таблицы, обеспечивает ссылочную целостность. Ограничивает: невозможно вставить несуществующую ссылку, удалить связанные записи (или CASCADE DELETE). Создает связи между таблицами (один-ко-многим, многие-ко-многим через junction table).

129. Что такое репликация и какие типы бывают?

Репликация — копирование данных с одного сервера (master/primary) на другие (slaves/replicas). Типы: 1) Синхронная — транзакция подтверждается после записи на реплики, надежнее, медленнее; 2) Асинхронная — запись сначала на master, затем на реплики, быстрее, риск потери данных. По топологии: Master-Slave (один пишет, многие читают), Master-Master (все пишут и читают), Multi-master. Цели: масштабирование чтения, отказоустойчивость, географическое распределение, резервное копирование.

130. Что такое шардирование (sharding)?

Шардирование — горизонтальное разделение данных между несколькими БД/серверами. Каждый шард содержит подмножество данных по sharding key (например, $user_id \% N$). Преимущества: масштабирование записи и чтения, распределение нагрузки. Недостатки: сложность JOIN между шардами, транзакции через шарды сложны, ребалансировка данных. Стратегии: Range-based (диапазоны ключей), Hash-based (хеш функции), Geographic (по географии), Directory-based (таблица маппинга).

131. В чем разница между SQL и NoSQL базами данных?

SQL (реляционные) — структурированные данные, схема, ACID, мощные JOIN, вертикальное масштабирование, строгая типизация. Примеры: PostgreSQL, MySQL, Oracle. NoSQL — гибкая схема, масштабирование горизонтальное, eventual consistency, примеры: MongoDB, Redis, Cassandra.

денормализация. Типы: Document (MongoDB, CouchDB), Key-Value (Redis, DynamoDB), Column-family (Cassandra, HBase), Graph (Neo4j). Выбор: SQL для сложных связей и транзакций, NoSQL для масштабирования и гибкости схемы.

132. Что такое CAP теорема?

CAP теорема — в распределенной системе невозможно одновременно гарантировать все три свойства: 1) Consistency (согласованность) — все узлы видят одинаковые данные; 2) Availability (доступность) — каждый запрос получает ответ (успех или ошибку); 3) Partition Tolerance (устойчивость к разделению) — система работает при сетевых разрывах. Нужно выбрать два из трех. На практике: CP (ACID БД), AP (eventual consistency NoSQL). При network partition выбор между C и A.

133. Что такое денормализация и когда ее применять?

Денормализация — намеренное добавление избыточности в нормализованную БД для улучшения производительности. Методы: дублирование данных, хранение агрегатов (сумм, счетчиков), материализованные представления, кеширование вычисляемых полей. Применять когда: читаемость критична (чтение >> записи), сложные JOIN замедляют запросы, агрегации выполняются часто. Компромисс: быстрее чтение, медленнее запись, риск рассинхронизации, сложнее поддержка целостности.

134. Что такое explain plan и для чего используется?

EXPLAIN (или EXPLAIN ANALYZE) — инструмент для анализа плана выполнения запроса. Показывает: какие индексы используются, порядок JOIN, типы сканирования (Sequential Scan, Index Scan), стоимость операций, количество строк. Используется для: оптимизации медленных запросов, проверки использования индексов, понимания узких мест. EXPLAIN показывает план, EXPLAIN ANALYZE выполняет и показывает реальное время. Ключевые метрики: cost, rows, execution time.

135. Что такое deadlock в БД и как его избежать?

Deadlock — ситуация, когда две или более транзакции взаимно блокируют друг друга, ожидая освобождения ресурсов. Пример: T1 блокирует A, ждет B; T2 блокирует B, ждет A. БД автоматически обнаруживает (timeout или граф ожидания) и откатывает одну транзакцию. Предотвращение: всегда блокировать ресурсы в одинаковом порядке, использовать короткие транзакции, минимизировать блокировки, использовать оптимистичные блокировки (versioning), установить lock timeout.

136. Что такое оптимистичная и пессимистичная блокировки?

Пессимистичная блокировка — блокирует данные при чтении до конца транзакции (SELECT FOR UPDATE). Предотвращает изменения другими транзакциями, гарантирует согласованность, но снижает конкурентность, риск deadlock. Оптимистичная — предполагает отсутствие конфликтов, проверяет перед записью (version/timestamp). Если данные изменились — откат и retry. Лучше для read-heavy, низкой конкуренции на запись. Пессимистичная для высокой конкуренции, критичных операций. Компромисс: блокировки vs производительность.

137. Что такое connection pool и зачем он нужен?

Connection pool — набор переиспользуемых соединений с БД. Вместо создания нового соединения на каждый запрос, берется из пула, используется, возвращается. Преимущества: быстрее (соединение дорогое: TCP handshake, аутентификация), ограничение нагрузки на БД, эффективное использование ресурсов. Параметры: min/max connections, idle timeout, connection lifetime, wait timeout. Неправильная настройка: мало соединений — bottleneck, много — перегрузка БД. Баланс зависит от workload и ресурсов БД.

138. Что такое MVCC?

MVCC (Multi-Version Concurrency Control) — механизм управления конкурентным доступом через версионирование данных. Каждая транзакция видит snapshot данных на момент начала. При изменении создается новая версия строки, старая сохраняется для других транзакций. Преимущества: читатели не блокируют писателей, писатели не блокируют читателей, высокая конкурентность. Используется в PostgreSQL, MySQL InnoDB, Oracle. Недостатки: overhead на хранение версий, требуется vacuum/purge для очистки старых версий.

139. Что такое Write-Ahead Logging (WAL)?

WAL (Write-Ahead Logging) — техника для обеспечения durability и atomicity. Принцип: изменения сначала записываются в лог (последовательная запись, быстро), затем применяются к данным (случайная запись, медленнее). При сбое: replay WAL для восстановления незафиксированных изменений. Преимущества: быстрые транзакции (отложенная запись данных), crash recovery, основа для репликации (streaming WAL). В PostgreSQL — WAL, MySQL — redo log, SQLite — journal. Компромисс: дополнительное I/O, но критично для надежности.

140. Что такое партиционирование (partitioning) таблиц?

Партиционирование — разделение большой таблицы на меньшие части (партиции) по правилу. Типы: Range (диапазоны значений, например даты), List (список значений), Hash (хеш функция), Composite (комбинация). Преимущества: быстрее запросы (сканируются только нужные партиции), легче удаление старых данных (drop partition), параллельные операции, улучшенное обслуживание (VACUUM, ANALYZE). Partition pruning — оптимизатор исключает ненужные партиции. Используется для больших таблиц (логи, временные ряды).

SOLID принципы

141. Что такое SOLID принципы и зачем они нужны?

SOLID — пять принципов объектно-ориентированного дизайна для создания поддерживаемого, гибкого и масштабируемого кода. Аббревиатура: S (Single Responsibility), O (Open/Closed), L (Liskov Substitution), I (Interface Segregation), D (Dependency Inversion). Цели: уменьшение связанности (coupling), увеличение связности (cohesion), упрощение тестирования, облегчение изменений и расширения. Применимы не только к ООП — в Go реализуются через интерфейсы, композицию и структуры.

142. Single Responsibility Principle (SRP) — что это и как реализуется в Go?

Принцип единственной ответственности: каждый модуль/тип должен иметь только одну причину для изменения, одну зону ответственности. В Go: структура или пакет решает одну задачу. Пример нарушения: структура `User` с методами для валидации, сохранения в БД, отправки email. Правильно: `User` — только данные, `UserValidator` — валидация, `UserRepository` — БД, `EmailService` — отправка. Преимущества: проще тестировать, легче изменять, лучше переиспользовать.

```
// Плохо: User делает всё
type User struct {
    Name string
    Email string
}

func (u *User) Validate() error { /* ... */ }
func (u *User) Save() error { /* ... */ }
func (u *User) SendEmail() error { /* ... */ }

// Хорошо: разделение ответственности
type User struct {
    Name string
    Email string
}

type UserValidator struct{}
func (v *UserValidator) Validate(u *User) error { /* ... */ }

type UserRepository struct{}
func (r *UserRepository) Save(u *User) error { /* ... */ }

type EmailService struct{}
func (s *EmailService) Send(to, subject, body string) error { /* ... */ }
```

143. Open/Closed Principle (OCP) — что это и как реализуется в Go?

Принцип открытости/закрытости: код должен быть открыт для расширения, но закрыт для модификации. В Go реализуется через интерфейсы: новый функционал добавляется через новые реализации интерфейса, а не изменением существующего кода. Пример: система обработки платежей — вместо if/switch для каждого типа, интерфейс `PaymentProcessor` с реализациями для разных методов оплаты. Добавление нового метода не требует изменения существующего кода.

```
// Плохо: нужно модифицировать при добавлении метода
func ProcessPayment(method string, amount float64) error {
    switch method {
        case "card":
            return processCard(amount)
        case "paypal":
            return processPaypal(amount)
    }
    // при добавлении нового метода нужно менять этот код
}

// Хорошо: открыто для расширения через интерфейс
type PaymentProcessor interface {
    Process(amount float64) error
}

type CardProcessor struct{}
func (p *CardProcessor) Process(amount float64) error { /* ... */ }

type PaypalProcessor struct{}
func (p *PaypalProcessor) Process(amount float64) error { /* ... */ }

type CryptoProcessor struct{} // новый тип без изменения существующего кода
func (p *CryptoProcessor) Process(amount float64) error { /* ... */ }

func ProcessPayment(processor PaymentProcessor, amount float64) error {
    return processor.Process(amount)
}
```

144. Liskov Substitution Principle (LSP) — что это и как реализуется в Go?

Принцип подстановки Барбары Лисков: объекты должны быть заменяемыми на экземпляры их подтипов без изменения правильности программы. В Go: любая реализация интерфейса должна выполнять контракт интерфейса корректно. Нарушение: если метод паникует, возвращает неожиданные значения или требует специальной обработки для конкретной реализации. Правильно: все реализации ведут себя предсказуемо согласно контракту интерфейса.

```

// Интерфейс для хранилища
type Storage interface {
    Save(key string, value []byte) error
    Load(key string) ([]byte, error)
}

// Хорошо: обе реализации корректно выполняют контракт
type FileStorage struct{}
func (s *FileStorage) Save(key string, value []byte) error {
    return os.WriteFile(key, value, 0644)
}
func (s *FileStorage) Load(key string) ([]byte, error) {
    return os.ReadFile(key)
}

type MemoryStorage struct {
    data map[string][]byte
}
func (s *MemoryStorage) Save(key string, value []byte) error {
    s.data[key] = value
    return nil
}
func (s *MemoryStorage) Load(key string) ([]byte, error) {
    val, ok := s.data[key]
    if !ok {
        return nil, os.ErrNotExist
    }
    return val, nil
}

// Плохо: ReadOnlyStorage нарушает контракт (Save всегда падает)
type ReadOnlyStorage struct{}
func (s *ReadOnlyStorage) Save(key string, value []byte) error {
    return errors.New("read-only storage") // нарушение LSP
}
func (s *ReadOnlyStorage) Load(key string) ([]byte, error) {
    return os.ReadFile(key)
}

// Правильно: отдельный интерфейс для read-only
type ReadOnlyStorageInterface interface {
    Load(key string) ([]byte, error)
}

```

145. Interface Segregation Principle (ISP) — что это и как реализуется в Go?

Принцип разделения интерфейсов: клиенты не должны зависеть от методов, которые они не используют. В Go: создавать маленькие, специфичные интерфейсы вместо больших универсальных. Go идиома: "accept interfaces, return structs" и "чем меньше интерфейс, тем он полезнее". Пример: вместо одного `Database` интерфейса с 20 методами — несколько маленьких: `Reader`, `Writer`, `Transactor`. Стандартная библиотека Go следует ISP: `io.Reader`, `io.Writer`, `io.Closer`.

```

// Плохо: большой интерфейс, клиенты зависят от ненужных методов
type Database interface {
    Connect() error
    Disconnect() error
    Query(sql string) ([]Row, error)
    Execute(sql string) error
    BeginTransaction() error
    Commit() error
    Rollback() error
    Backup() error
    Restore() error
}

// Функция нуждается только в чтении, но зависит от всего интерфейса
func GetUsers(db Database) ([]User, error) {
    rows, err := db.Query("SELECT * FROM users")
    // ...
}

// Хорошо: разделенные интерфейсы
type Querier interface {
    Query(sql string) ([]Row, error)
}

type Executor interface {
    Execute(sql string) error
}

type Transactor interface {
    BeginTransaction() error
    Commit() error
    Rollback() error
}

// Функция зависит только от того, что использует
func GetUsers(q Querier) ([]User, error) {
    rows, err := q.Query("SELECT * FROM users")
    // ...
}

// Композиция интерфейсов для комплексных операций
type Database interface {
    Querier
    Executor
    Transactor
}

// Примеры из стандартной библиотеки Go (ISP в действии)
// io.Reader, io.Writer, io.Closer – маленькие, специфичные интерфейсы
func ProcessData(r io.Reader) error {
    // нужен только Reader, не зависит от Writer или Closer
}

```

146. Dependency Inversion Principle (DIP) — что это и как реализуется в Go?

Принцип инверсии зависимостей: модули высокого уровня не должны зависеть от модулей низкого уровня, оба должны зависеть от абстракций. В Go: зависимость от интерфейсов, а не конкретных реализаций. Dependency Injection через конструкторы или параметры функций. Пример: сервис зависит от интерфейса `Repository`, а не конкретной `PostgresRepository`. Это позволяет легко менять реализацию и мокать при тестировании.

```

// Плохо: зависимость от конкретной реализации
type UserService struct {
    db *PostgresDatabase // зависит от конкретной реализации
}

func NewUserService() *UserService {
    return &UserService{
        db: &PostgresDatabase{}, // жесткая связь
    }
}

func (s *UserService) GetUser(id int) (*User, error) {
    return s.db.FindUserByID(id)
}

// Хорошо: зависимость от абстракции (интерфейса)
type UserRepository interface {
    FindByID(id int) (*User, error)
    Save(user *User) error
}

type UserService struct {
    repo UserRepository // зависит от интерфейса
}

// Dependency Injection через конструктор
func NewUserService(repo UserRepository) *UserService {
    return &UserService{
        repo: repo,
    }
}

func (s *UserService) GetUser(id int) (*User, error) {
    return s.repo.FindByID(id)
}

// Конкретные реализации
type PostgresRepository struct {
    db *sql.DB
}

func (r *PostgresRepository) FindByID(id int) (*User, error) {
    // PostgreSQL implementation
}

type MongoRepository struct {
    client *mongo.Client
}

func (r *MongoRepository) FindByID(id int) (*User, error) {
    // MongoDB implementation
}

// Легко менять реализацию
func main() {
    // Production
    postgresRepo := &PostgresRepository{db: db}
    service := NewUserService(postgresRepo)

    // Testing
    mockRepo := &MockRepository{}
    testService := NewUserService(mockRepo)
}

// Mock для тестирования
type MockRepository struct {
    users map[int]*User
}

func (r *MockRepository) FindByID(id int) (*User, error) {
    if user, ok := r.users[id]; ok {
        return user, nil
    }
    return nil, errors.New("not found")
}

```

147. Как SOLID принципы помогают в тестировании Go кода?

SOLID принципы делают код легко тестируемым: 1) SRP — изолированные компоненты легко тестировать по отдельности; 2) OCP — новые тесты без изменения существующих; 3) LSP — мокать любую реализацию интерфейса; 4) ISP — мокать только нужные методы, минимальные зависимости; 5) DIP — легко подменять зависимости на моки через интерфейсы. Dependency Injection позволяет передавать тестовые реализации. В Go: моки через интерфейсы, table-driven tests, пакеты testify/mock или gomock для генерации моков.

```
// Благодаря DIP и ISP легко тестировать
type UserService struct {
    repo UserRepository
    emailService EmailSender // маленький интерфейс
}

type EmailSender interface {
    Send(to, subject, body string) error
}

func (s *UserService) RegisterUser(email string) error {
    user := &User{Email: email}
    if err := s.repo.Save(user); err != nil {
        return err
    }
    return s.emailService.Send(email, "Welcome", "Thanks for registering")
}

// Тестирование с моками
func TestRegisterUser(t *testing.T) {
    // Моки реализуют те же интерфейсы
    mockRepo := &MockRepository{
        users: make(map[int]*User),
    }
    mockEmail := &MockEmailSender{
        sentEmails: []string{},
    }

    service := NewUserService(mockRepo, mockEmail)

    err := service.RegisterUser("test@example.com")
    assert.NoError(t, err)
    assert.Equal(t, 1, len(mockEmail.sentEmails))
}

type MockEmailSender struct {
    sentEmails []string
}

func (m *MockEmailSender) Send(to, subject, body string) error {
    m.sentEmails = append(m.sentEmails, to)
    return nil
}
```

148. Какие компромиссы и проблемы могут возникать при применении SOLID?

Чрезмерное применение SOLID может привести к: 1) Over-engineering — слишком много мелких интерфейсов и типов для простых задач; 2) Снижение производительности — дополнительные уровни абстракции, indirect calls через интерфейсы; 3) Усложнение кода — труднее следить за потоком выполнения; 4) Преждевременная абстракция — создание интерфейсов "на будущее". Баланс: начинать с простого решения, рефакторить при появлении реальной необходимости. "Make it work, make it right, make it fast". В Go: избегать интерфейсов пока не нужны две реализации или моки для тестов.

```
// Over-engineering: слишком много абстракций для простой задачи
type ConfigReader interface {
    Read() (Config, error)
}

type ConfigValidator interface {
    Validate(Config) error
}

type ConfigLoader interface {
    Load() (Config, error)
}

type ConfigLoaderImpl struct {
    reader ConfigReader
    validator ConfigValidator
}

// Для простого случая достаточно одной функции
func LoadConfig(path string) (Config, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return Config{}, err
    }
    var cfg Config
    if err := json.Unmarshal(data, &cfg); err != nil {
        return Config{}, err
    }
    return cfg, nil
}

// Правило: "Don't create abstractions, discover them"
// Создавать интерфейсы когда появляется вторая реализация или нужны моки
```

149. Что такое Apache Kafka и для чего используется?

Kafka — распределённый брокер сообщений (distributed commit log). Сообщения пишутся в топики и хранятся на диске заданное время (retention), а не удаляются сразу после чтения. Используется для: event streaming (потоки событий между сервисами), асинхронной обработки, CDC (Change Data Capture, например Debezium), логов аналитики, буферизации пиков нагрузки. Высокая пропускная способность, горизонтальное масштабирование через партиции, отказоустойчивость через репликацию.

150. Из каких компонентов состоит Kafka?

Broker — сервер, хранящий данные топики. **Topic** — именованный поток сообщений. **Partition** — единица параллелизма и хранения внутри топика. **Producer** — публикует сообщения. **Consumer** — читает сообщения. **Consumer group** — группа консьюмеров, совместно обрабатывающих топик (каждая партиция — одному консьюмеру группы). **ZooKeeper** (старые версии) или **KRaft** (Kafka 3.3+, по умолчанию с 4.0) — координация кластера, метаданные, выбор лидеров. **Schema Registry** (опционально, Confluent) — хранение схем Avro/Protobuf/JSON.

151. Что такое topic, partition и offset?

Topic — логическая категория сообщений (например `orders.created`). **Partition** — физический упорядоченный лог внутри топика; сообщения в партиции имеют строгий порядок. Партиции распределены по брокерам — это даёт параллелизм. **Offset** — монотонно растущий номер сообщения внутри партиции (не глобальный). Консьюмер отслеживает offset — «до какого места прочитал». Offset коммитится в специальный топик `__consumer_offsets` или во внешнее хранилище.

152. Как обеспечивается порядок сообщений в Kafka?

Порядок гарантирован **только внутри одной партиции**. Если нужен порядок для сущности (заказ, пользователь) — все её события должны попадать в одну партицию через **partition key** (producer указывает ключ, хеш ключа → номер партиции). Больше партиций = больше параллелизма, но порядок между партициями не гарантируется. Для глобального порядка — одна партиция (узкое место).

153. Что такое consumer group и как работает rebalance?

Consumer group — набор консьюмеров с одним `group.id`, совместно читающих топик. Каждая партиция назначается **ровно одному** консьюмеру группы. При добавлении/удалении консьюмера или изменении партиций происходит **rebalance** — перераспределение партиций. Протоколы: Range, RoundRobin, Cooperative (sticky) — последний минимизирует «stop-the-world» при rebalance. Во время rebalance обработка останавливается — частая причина лагов. `max.poll.interval.ms` и `session.timeout.ms` влияют на детект «мёртвых» консьюмеров.

154. Как устроена репликация в Kafka? Что такое ISR?

Каждая партиция имеет **replication factor** копий на разных брокерах. Одна реплика — **leader** (принимает read/write), остальные — **followers** (реплицируют). **ISR** (In-Sync Replicas) — реплики, не отстающие от лидера больше чем на `replica.lag.time.max.ms`. Запись считается подтверждённой при `acks=all` (или `-1`) когда все ISR записали. При падении лидера новый выбирается из ISR. `min.insync.replicas` — минимум ISR для записи; если меньше — producer получит ошибку (защита от потери данных).

155. Чем отличаются at-least-once, at-most-once и exactly-once?

At-most-once — сообщение может потеряться, но не продублируется. Producer: `acks=0` или `fire-and-forget`; consumer коммитит offset до обработки. **At-least-once** — сообщение не теряется, но может обработаться повторно. Producer ждёт `ack`; consumer коммитит offset **после** обработки. Нужна идемпотентность на стороне обработчика. **Exactly-once** — каждое сообщение обрабатывается ровно один раз. В Kafka: идемпотентный producer + транзакции (read-process-write в одной транзакции) или EOS в Kafka Streams. Дороже по latency и сложнее в настройке.

156. Что такое идемпотентный producer?

Producer с `enable.idempotence=true` получает уникальный **Producer ID (PID)** и для каждой партиции ведёт счётчик `sequence number`. Брокер отбрасывает дубликаты при ретрайах (сетевой сбой → повторная отправка). Гарантирует отсутствие дублей **внутри одной сессии producer** на уровне записи в партицию. Не заменяет идемпотентность consumer — если сообщение уже обработано, а offset не закоммичен, consumer прочтает его снова. Требуется `acks=all`, `retries > 0`, `max.in.flight.requests.per.connection <= 5`.

157. Как работают Kafka transactions?

Транзакции позволяют атомарно: записать в несколько партиций/топики + закоммитить consumer offsets в одной транзакции. Producer вызывает `initTransactions()` → `beginTransaction()` → `send/consumer offsets` → `commitTransaction()` или `abortTransaction()`. Брокер пишет маркеры транзакций (control records). Консьюмер с `isolation.level=read_committed` видит только закоммиченные сообщения. Используется для exactly-once семантики в пайплайнах обработки. Overhead: координатор транзакций, таймаут `transaction.timeout.ms`.

158. В чем разница между Kafka и RabbitMQ?

Kafka — log-based, сообщения хранятся (retention), consumer сам тянет (pull), высокий throughput, replay (перечитать с любого offset). **RabbitMQ** — queue-based, сообщение удаляется после `ack` (по умолчанию), push-модель, гибкая маршрутизация (exchanges, routing keys), лучше для task queues и RPC. Kafka — event streaming, аналитика, CDC. RabbitMQ — классические очереди задач, сложная маршрутизация, меньшие объёмы. В Go: `segmentio/kafka-go`, `confluent-kafka-go` (librdkafka) для Kafka; `amqp091-go` для RabbitMQ.

159. Что такое log compaction?

Политика хранения топика: вместо удаления по времени/размеру брокер оставляет **последнее значение для каждого ключа**. Старые записи с тем же ключом удаляются. Используется для changelog-топики (состояние сущностей, конфиги, Kafka Connect).

offsets). Топик остаётся компактным, но содержит актуальный снимок по каждому ключу. Не подходит для событий, где нужна полная история — там retention по времени.

160. Что такое KRaft и чем он заменил ZooKeeper?

KRaft (Kafka Raft) — встроенный механизм консенсуса на базе Raft для хранения метаданных кластера (топики, партиции, ACL, конфигурация). Заменяет внешний **ZooKeeper**, который раньше был обязательной зависимостью. Плюсы: проще деплой (меньше компонентов), быстрее failover контроллера, масштабирование до миллионов партиций. С Kafka 4.0 KRaft — единственный режим (ZooKeeper удалён). Миграция: rolling upgrade с режима ZK на KRaft.

161. Как спроектировать схему топиков для микросервисов?

Принципы: **событие как факт** (`order.created`, не `updateOrder`); топик на тип события или bounded context; ключ партиции = ID сущности для порядка; отдельные топика для разных consumers (не фильтровать в consumer то, что можно разделить топиками); schema evolution через Avro/Protobuf + Schema Registry (backward/forward compatibility). Паттерны: **Event-Carried State Transfer**, **CQRS** (отдельные топика для команд и проекций), **Outbox** (запись в БД + outbox таблица → Debezium/relay в Kafka для гарантии доставки). Избегать god-topic с тысячей типов событий.

162. Что такое consumer lag и как с ним бороться?

Lag — разница между последним offset в партиции (log end offset) и offset, закоммитенным consumer group. Растущий lag = consumer не успевает. Причины: медленная обработка, rebalance, мало партиций (мало параллелизма), GC/long pauses, `max.poll.records` слишком большой. Решения: увеличить партиции и инстансы consumer (до числа партиций), оптимизировать handler, batch-обработка, асинхронная обработка с отдельным коммитом, мониторинг через Kafka Exporter / Burrow / встроенные метрики. Lag по партиции — признак неравномерного распределения ключей (hot partition).

163. Как работает CDC через Kafka (Debezium)?

CDC (Change Data Capture) — захват изменений из БД и публикация в Kafka. **Debezium** читает WAL/binlog БД (PostgreSQL, MySQL и др.), не нагружая таблицы polling-ом. Каждая строка → событие с op (create/update/delete), до/после состояние. Топики обычно `server.schema.table`. Используется для: синхронизации read-моделей, event-driven архитектуры, миграции данных (Strangler Pattern). Важно: ordering по primary key, идемпотентные consumers, обработка snapshot + streaming фазы, схема при delete (tombstone + compaction).